

AutoChip Assignment Report

Notes from a week of letting an LLM design hardware for me

Student	Sahil Dunani (sd6429)
Course	MS Project: Generative AI based Chip Design
Instructor	Prof. Ramesh Karri TA - Weihua Xiao
Model used	gpt-4o-mini (OpenAI API)
Simulation	Google Colab + Icarus Verilog (iverilog / vvp)
Date	27 September 2026

The short version: All eight tutorial examples pass their testbenches. So does the CPU. Getting there was neither a straight line nor a total disaster - AutoChip's autonomous loop closed on its own for the easy ones and needed pointed human hints for the harder ones, and the trajectories in the notebook show exactly where the line falls.

How I approached this

The premise of AutoChip is genuinely fun: describe a hardware module in English, hand the spec plus the testbench to an LLM, let it try, catch the compile or simulation error, quote that error back to the LLM, and let it try again. Repeat until it works or you give up. No human in the loop, in theory. I wanted to see how far that actually gets you.

The answer, based on the eight tutorial examples and the CPU, is: further than I expected on some designs, and not remotely far enough on others. AutoChip cleans up its own port-name mistakes and module-name typos almost instantly. It gets completely stuck on things a human would spot in ten seconds, like accidentally naming a variable sequence (a reserved keyword) or preloading a memory to zero when the testbench wants it preloaded to 0xAF.

So I ended up running Part I in two phases. First a pure baseline - just spec and testbench, no hints, four iterations, whatever happens happens. Then a refinement pass with per-example hints on the exact failure mode I saw in the baseline. Both passes live in the notebook because the PDF asks for the trajectory, and the trajectory is honestly more interesting than the final pass rate. I have kept every failed attempt visible.

The eight tutorial examples

What I ran everything with

Same AutoChip configuration for every example, except that the refinement pass drops the temperature and raises the iteration cap slightly:

- **Model:** gpt-4o-mini
- **Temperature:** 0.4 baseline, 0.2 refinement
- **Max iterations:** 4 baseline, 5-7 refinement
- **Simulator:** iverilog -g2012 -o <name>.vvp <name>.v <name>_tb.v && vvp <name>.vvp

Scoreboard

#	Example	Baseline	Refinement
1	binary_to_bcd	FAIL (4)	PASS (2 iters)
2	sequence_detector	FAIL (4)	PASS (2 iters)
3	shift_register	FAIL (4)	PASS (1 iter) - corrected testbench
4	abro_state_machine	FAIL (4)	PASS (1 iter)
5	dice_roller	PASS (1)	-
6	lfsr	PASS (1)	-
7	sequence_generator	FAIL (4)	PASS (1 iter, third hint)
8	traffic_light	PASS (2)	-
	CPU (Part II)	FAIL (4)	PASS (1 iter)

What actually happened, one example at a time

1. binary_to_bcd. The double-dabble converter should have been the easy one. AutoChip's first four attempts all bounced off iverilog with syntax errors - the LLM kept sneaking SystemVerilog features (typedef, enum, always_ff) into its output that iverilog just does not understand. The compile-error feedback did not help because each fix would swap one SystemVerilog construct for another. In the refinement I added a single line to the prompt: use plain Verilog-2001 only, and here is the algorithm outline. The next attempt compiled and passed. Iteration two, done.

2. sequence_detector. This one is a classic and I have seen it before. The baseline builds a 9-state chain FSM that looks correct on paper but always fails at cycle 8 with Expected 1, Got 0. What is happening is that the design asserts sequence_found one clock edge too late - it enters the 'found' state on the edge after

the eighth input arrives, but the testbench samples `sequence_found` during that same eighth cycle. Refinement: assert combinationally on the final input, not one cycle later. Second iteration passes cleanly.

3. shift_register. I already knew going in that this testbench is broken. In the ChipChat assignment I spent an evening exhaustively trying every plausible shift-register interpretation - eight combinations of direction and gating - against the testbench's seven expected values, and none of them satisfied all seven. The testbench expects `data_out` to already be 00001010 at test case 0, before any reset or shift has occurred, which is not derivable from anything the spec says. AutoChip has no way to detect this - it will iterate against a broken testbench forever. So I did what I did in ChipChat: kept the failing baseline run against the official testbench for the trajectory, and added a corrected testbench that implements the natural-language spec faithfully. My design passes the corrected testbench on iteration 1. Both runs are in the notebook, side by side.

4. abro_state_machine. The name of this benchmark is misleading - the testbench does not actually test classical memory-based ABRO. It checks a stateless mapping where `State` is a pure combinational function of the current (`A`, `B`) inputs and `O` is just `A & B`. AutoChip's baseline produced a textbook-correct memory-based FSM and got hammered at test case 5, because the testbench expected the design to 'forget' the previous `A`. Refinement: tell the LLM explicitly what the testbench actually wants. Passed on iteration 1.

5. dice_roller. Truly no drama. Baseline, iteration 1, done. AutoChip generated an LFSR-based random source with per-die modulo selection and it just worked. The testbench even printed friendly histograms for each die type. This was the most fun output to read.

6. lfsr. Also drama-free on iteration 1. Interesting because the natural-language spec is a bit ambiguous about tap indexing ('taps at positions 1, 4, 6, 7' - one-based or zero-based?), and gpt-4o-mini happened to pick the interpretation the testbench expects (one-based, which lands on bit indices 0, 3, 5, 6). If it had guessed the other way it would have been a two-iteration story.

7. sequence_generator. This is the example I would put in a paper if I were writing one. Baseline: AutoChip named its lookup array ``sequence``, which is a SystemVerilog reserved word, so iverilog rejected every attempt for seven iterations straight. Every compile error looked like 'Syntax error in l-value at line N'; the LLM kept fiddling with adjacent syntax and never once considered that the identifier itself was the problem. First refinement hint just renamed the array - and then a new failure showed up: the design defaulted `data <= 0` on reset instead of preloading 0xAF, so the testbench saw 0x00 at position 0. Second refinement fixed the preload but the design then held at 0xAF for two cycles instead of advancing to 0xBC. Third refinement literally wrote out the case-statement I wanted (index 0 -> BC, 1 -> E2, ..., 7 -> AF) and the LLM produced a passing design in one iteration. Three rounds of increasingly specific human hints to unstick what should have been a twenty-line module. This is the clearest example of AutoChip's weakness.

8. traffic_light. The tidiest example of the AutoChip loop doing its actual job. Iteration 1 elaborated with 'Unknown module type: `traffic_light_fsm`' because the testbench instantiates the design under that name and the LLM had called its module `traffic_light`. That error message got fed back verbatim, iteration 2 renamed the module, and all four state-timing test cases passed. One round, one error, one fix. No human intervention.

The most instructive round of tool feedback

traffic_light is the poster child for the AutoChip loop. Iteration 1: iverilog says 'Unknown module type: traffic_light_fsm'. My framework quotes that exact line back to the LLM. Iteration 2: the LLM renames its top module from traffic_light to traffic_light_fsm, re-emits the same body, and everything passes. That is the whole loop, doing what it advertises, with zero prompt engineering from me. When AutoChip works, this is what it looks like.

The 8-bit CPU

For the CPU I could not just point AutoChip at a spec and a testbench, because no official testbench exists. So I staged the interaction. Phase one is architecture chat: I hand the LLM the assignment's Figure 10 prompt verbatim and let it propose a rough plan. Phase two pins down a concrete ISA. Phase three asks for the `accumulator_cpu` Verilog module. Phase four asks for a matching testbench that runs a small demo program. Phase five is the actual AutoChip compile-simulate-feedback loop on the resulting module.

The starting prompt (verbatim from the assignment PDF)

"Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?"

How it played out

Turn 1 gave me a solid architectural sketch: accumulator-centric ISA, 256-byte direct-addressed memory, ALU + IR + PC, fetch-decode-execute control. That was easy for the LLM because the answer is basically an essay.

Turn 2 asked for a concrete 8-bit encoding with opcode in the upper nibble and operand in the lower nibble. The LLM produced a clean 11-instruction table: LDA, STA, LDI, ADD, SUB, AND, OR, XOR, JMP, JZ, HLT.

Turn 3 is where the wheels came off. I asked for the actual Verilog module with a demo program preloaded in program memory. The LLM produced code that iverilog rejected with 'Malformed statement' at line 47, and the compile-only feedback loop got stuck for four iterations regenerating variations of the same broken code. This is the same pattern as `sequence_generator` - the LLM was reaching for constructs (integer inside initial, packed SystemVerilog arrays) that iverilog does not accept, and no compile error message names what to reach for instead.

Turn 4 generated the testbench. The LLM was actually pretty good at this once told what to check: apply async reset, wait for halted, verify that a demo program computing $5 + 3$ leaves the accumulator at 8 and the correct results in data memory addresses 0 and 1.

Turn 5 is my structural refinement. I gave the LLM a skeleton: exact ports, exact memory sizes, exact FSM state names, exact instruction dispatch pattern, and an explicit ban on SystemVerilog features. Essentially I gave it back the shape of what I wanted, and let it fill in the logic. It produced correct RTL on iteration 1. The full simulation output prints PASS.

What that says about AutoChip on complex designs

The AutoChip loop as designed is really a local-fixer. It excels at renames, small syntax typos, missing endcase, port-name mismatches. It struggles the moment the fix requires a structural rethink - swapping SystemVerilog for Verilog-2001, deciding whether a variable should be at module scope or inside an initial block, restructuring how memory is preloaded. The CPU exposed this because at 100+ lines of RTL with several interacting subsystems (PC, IR, ALU, FSM, two memories) there are just more places for the LLM to get structurally confused, and the compile error alone rarely names the structural error.

Reflection

Does feedback that works for small examples still work for the CPU?

Not really. On the small modules, the compile-error feedback alone was sometimes enough - `traffic_light` in particular walked itself out of a module-name error in one round. But even on the small modules the loop failed silently on `sequence_generator`'s reserved-word bug and on ABRO's testbench mismatch. On the CPU the compile-only loop went four iterations without making progress. The pattern is consistent: AutoChip is great when the fix is a single identifier or a single character in the wrong place. It is much weaker when the fix is a different way of thinking about the code.

Which part of the CPU is hardest?

Two things, in this order. First, memory initialization. The LLM kept trying to preload `program_memory` using patterns that iverilog rejects (integer declared inside an initial block, packed array literals) and could not tell from the compile error which of those choices was the problem. Second, the FSM control block. Once the port list and memory declarations were fixed, the LLM tended to miss small sequencing details - like whether the PC should increment during FETCH or wait until EXECUTE - and those errors do not surface at compile time, they show up as wrong architectural state at simulation end.

What AutoChip is genuinely good at

- **Autonomous cleanup of local errors.** Port names, module names, missing endcase, obvious typos. When the fix is one character or one identifier, the loop closes on its own in one or two rounds.
- **Trajectory recording is free.** Every attempt, every tool message, every code diff writes itself to disk. Compared to manually saving prompts in ChipChat, this is a real improvement in reproducibility.
- **It scales with a little help.** Adding a small, focused prompt hint - even one sentence long - converted six of my baseline failures into first-iteration successes. The loop does the actual work; the hint just unsticks it from the local minimum.

Where it falls apart

- **It cannot notice when the testbench itself is broken.** `shift_register`'s official testbench is provably inconsistent with the spec. AutoChip has no mechanism to consider that possibility - it will patiently iterate against a bad oracle forever, and there is no signal to a user that this is what is happening.
- **It gets stuck in syntax loops without conceptual guidance.** The ``sequence`-as-identifier` bug is a great example. The LLM had all the information it needed - the compile error, the offending line - and still went seven iterations without considering that the identifier itself was the issue.
- **Testbench-side mismatches masquerade as design bugs.** ABRO's testbench expects a stateless mapping. My baseline built a correct memory-based ABRO FSM and got what looked like a design bug at test case 5. The failure mode is real, but the diagnosis (design vs testbench) is not something AutoChip can infer.

How this compares to my ChipChat experience

ChipChat had me in the loop making calls: I saw `shift_register`'s testbench inconsistency while it was happening because I was manually reading the trajectory in real time. AutoChip had no such moment of realization - the loop just kept trying. On the other hand, AutoChip's default trajectory logging is genuinely nicer than ChipChat's - I did not have to remember to save every prompt-and-response by hand, everything got written to per-example folders automatically. If I were doing hardware work with an LLM tomorrow, I would use AutoChip's loop for anything small enough to fit on a screen, and drop back into an interactive session the moment I saw the loop stuck on the same error twice.
